

Confluence Cloud REST API v2 – Comprehensive Endpoint Overview

Admin Key

- **GET** `/admin-key` – **Get Admin Key**: Checks if an admin key is currently enabled for the calling user (available only to site/organization admins on Confluence Cloud Premium/Enterprise) ¹ ². Requires the user to be a site or org admin; Forge/OAuth2/Connect apps cannot access this resource ³. Returns the admin key's accountId and expiration time if one is active ⁴ ⁵.
- **POST** `/admin-key` – **Enable Admin Key**: Issues a new admin key for the calling admin user (or renews it if one exists) ⁶. Optionally accept a JSON body with `"durationInMinutes"` (default 10 minutes if not provided) ⁷. Requires admin privileges as above. Returns the new key's accountId and expiration time on success ⁸.
- **DELETE** `/admin-key` – **Disable Admin Key**: Revokes the admin key for the calling user (if one exists) ⁹. Requires admin privileges. Returns 204 No Content on success (also returns 204 if no key was enabled to begin with) ¹⁰.

Attachment

- **GET** `/attachments` – **List attachments**: Retrieves all attachments across the site, limited by `limit` (with pagination via `cursor` and `Link: next` header) ¹¹ ¹². Supports query params to filter by `sort` (order by created/modified date ¹³), `status` (e.g. current/trashed), `mediaType` (content type), `filename`, etc. ¹⁴ ¹⁵. Returns a paginated list of attachments with fields like `id`, `status`, `title`, `createdAt`, `pageId`, `blogPostId`, `customContentId` (container), `mediaType`, `fileSize`, `comment`, `downloadLink`, and version info ¹⁶.
Permissions: user must have view access to the attachments' parent content ¹⁷.
- **GET** `/attachments/{id}` – **Get attachment by ID**: Returns metadata for a specific attachment (identified by its ID) ¹⁸. Optional query params `version` (to get a specific version's metadata), and boolean flags like `include-versions`, `include-properties`, `include-labels`, etc. to expand details ¹⁹ ²⁰. By default returns the latest version info. **Permissions**: Requires view permission on the attachment's parent content ²¹.
- **DELETE** `/attachments/{id}` – **Delete attachment**: Removes an attachment (moves to trash by default, or permanently deletes if the space recycle bin is purged) ²². **Permissions**: Requires permission to delete attachments in that space (and space admin if attempting a purge) ²³.
- **GET** `/blogposts/{id}/attachments` – **List attachments on a blog**: Retrieves attachments of a specific blog post ²⁴. Supports `limit` /pagination, and same filters as general list. **Permissions**: Need view permission on that blog post ¹⁷.
- **GET** `/custom-content/{id}/attachments` – **List attachments on custom content**: Retrieves attachments on a piece of custom content ²⁵. (Custom content refers to app-defined content types.) Same parameters and permission requirements as above.

- **GET** `/labels/{id}/attachments` – **List attachments by label:** Retrieves attachments that have a given label ²⁶. Provide the Label ID; supports pagination. Only attachments the user can view are returned.
- **GET** `/pages/{id}/attachments` – **List attachments on a page:** Retrieves attachments on a specific page ²⁷. Similar behavior to blog post attachments. (All these attachment-list endpoints share the same structure and pagination behavior.)

App Properties

- **GET** `/app/properties` – **Get Forge app properties:** Returns all properties stored for the calling Forge app (these are key-value data specific to an app) ²⁸ ²⁹. *Forge only:* this endpoint can only be called from a Forge app's backend using the Forge bridge (not via user credentials) ³⁰. Pagination is supported via `limit` and `cursor`. **Scopes:** requires `read:app-data:confluence` OAuth scope ³¹. Returns an array of `{ "key": ..., "value": ... }` objects ³².
- **GET** `/app/properties/{propertyKey}` – **Get app property by key:** Retrieves the value of a specific app property by its key ³³. Forge-only as above ³⁴. Returns a JSON object with the property's key and value on success ³⁵ (404 if not found).
- **PUT** `/app/properties/{propertyKey}` – **Create/Update app property:** Sets the value of a Forge app property (creates if it doesn't exist, or updates if it does) ³⁶. Request body is a JSON object representing the new value. **Scopes:** requires `write:app-data:confluence` ³⁷. Returns 200 if updated or 201 if created ³⁸.
- **DELETE** `/app/properties/{propertyKey}` – **Delete app property:** Removes the specified property. (Not explicitly shown above, but the operations list indicates this exists ³⁹.) It would require the same app context and appropriate scope.

Ancestors (Content Ancestor Tree)

- **GET** `/whiteboards/{id}/ancestors` – **Get ancestors of a whiteboard:** Returns the hierarchy of parent pages/folders up to the root for the given whiteboard (ordered from highest ancestor down to its direct parent) ⁴⁰ ⁴¹. Useful to determine where a whiteboard resides in the space content tree. Limited by `limit` with pagination – if more ancestors exist (which is uncommon), you'd call the endpoint again with the last returned ancestor's ID to get higher levels ⁴⁰. **Permissions:** User must have view access to the whiteboard and all ancestor pages/spaces ⁴².
- **GET** `/databases/{id}/ancestors` – **Get ancestors of a database:** Similarly returns the parent chain for a database content item ⁴³. Follows the same behavior and requirements as above.
- **GET** `/embeds/{id}/ancestors` – **Get ancestors of a Smart Link embed:** Returns the ancestors for a Smart Link (an embedded object in the content tree) ⁴⁴. This shows where in the hierarchy the smart link resides (e.g. parent page).
- **GET** `/folders/{id}/ancestors` – **Get ancestors of a folder:** Returns the ancestor path of a folder (folders are new structured content containers) ⁴¹. For example, parent folders and ultimately the space.
- **GET** `/pages/{id}/ancestors` – **Get ancestors of a page:** Returns all ancestor pages of the given page, up to the space root ⁴⁵. The highest-level ancestor (usually the space home or a top-level page) is first in the list. **Note:** All these "ancestors" endpoints only return direct lineage (no siblings or children). Pagination generally isn't needed unless the content is deeply nested.

(All “ancestors” endpoints share the same format: the response is a list of ancestor objects with their IDs and titles. Only content the user can view will appear.)

Blog Post

- **GET** `/blogposts` – **List blog posts**: Retrieves all blog posts the user can view, across the site ⁴⁶. Results are sorted by ID (ascending) by default or can be sorted by other fields like title or created date via the `sort` parameter (e.g. “created-date-newest-to-oldest”) ⁴⁷ ⁴⁸. Supports pagination with `limit` and `cursor`. You can also filter by space (`space-id` query param), status (current/trashed), etc. Returns each blog post’s details (ID, title, space, status, author, etc.). **Permissions**: Only blog posts the user has permission to view are returned ⁴⁹.
- **POST** `/blogposts` – **Create blog post**: Creates a new blog post in a specified space. The request body must include at least a title and body content. You can also specify the space via its `spaceId` or `spaceKey` and other optional fields (like `status` if creating as draft). **Permissions**: User needs “Add blog” permission in the target space ⁵⁰ ⁵¹. On success, returns 201 and the created blog post object. (If the request is missing required fields or the user lacks permission, returns 400 or 401.)
- **GET** `/blogposts/{id}` – **Get blog post by ID**: Retrieves a specific blog post’s details by its ID. By default returns the current version content. You can expand certain fields via query (e.g. `body-format` to get storage or view format content, `include-attachments`, `include-operations` to see allowed operations, etc.). **Permissions**: Requires view access to that blog’s space ⁵² ⁵³.
- **PUT** `/blogposts/{id}` – **Update blog post**: Updates an existing blog post’s content or metadata. Typically, you would supply a new body and optionally a new title (or use the separate title endpoint). A new version is created. The request body for update must include a `version` object with an incremented version number ⁵⁴. **Permissions**: Requires permission to edit blogs in that space ⁵³. On success, returns the updated blog post. (Minor edits can be indicated in the version object if supported.)
- **DELETE** `/blogposts/{id}` – **Delete blog post**: Deletes the specified blog post. This moves the blog to trash if trash is enabled (status becomes “trashed”). Only space admins (or users with delete permission) can do this ⁵⁵ ⁵⁶. Returns 204 on success.
- **GET** `/labels/{id}/blogposts` – **Get blog posts for label**: Retrieves all blog posts that have a given label. Provide the Label ID (or name, depending on implementation). Supports `limit` / pagination. **Permissions**: Only blog posts visible to the user are returned.
- **GET** `/spaces/{id}/blogposts` – **Get blog posts in space**: Retrieves all blog posts within a specific space (space ID or key). Supports pagination and similar filters (e.g. limit, status). This is useful to list blogs space by space. **Permissions**: User must have view access to the space. (Blog post objects include fields like `id`, `title`, `status` (current or trashed), `spaceId`, `createdAt`, `updatedAt`, `history` (version info), and body content in various formats. The API uses cursor pagination for listing.)

Children (Direct Children of Content)

These endpoints retrieve the **direct child items** under a specified content item (not all descendants, only one level down). They are grouped by content type:

- **GET** `/whiteboards/{id}/direct-children` – **Direct children of whiteboard**: Returns the immediate children of a whiteboard (if any). In Confluence, a whiteboard might not typically have child content, so this may often return an empty list.

- **GET** `/databases/{id}/direct-children` - **Direct children of database**: Returns immediate children of a database item. (A “database” could contain child pages or other nested content in the hierarchy.)
- **GET** `/embeds/{id}/direct-children` - **Direct children of Smart Link**: Returns content items directly contained under a Smart Link embed. (Likely rarely used since Smart Links are usually leaves, not parents, but the endpoint exists for completeness.)
- **GET** `/folders/{id}/direct-children` - **Direct children of folder**: Returns the list of content items directly within the given folder ⁵⁷ ⁵⁸. Folders can contain pages, whiteboards, databases, etc., so this could return a mix of types (each child item will have an `objectType` field indicating if it’s a page, etc.).
- **GET** `/pages/{id}/children` - **Children of page**: Returns all immediate children of a page, **including all content types** (pages, but also any folders, whiteboards, etc. directly under that page) ⁵⁹. This is a **non-filtered** children listing. (Note: V2 differentiates direct page children by type; see next item.)
- **GET** `/pages/{id}/direct-children` - **Direct children of page (pages only)**: Returns immediate children that are *pages* specifically ⁵⁸. This endpoint is more specialized: it will list only child pages under the given page (excluding other content types). In contrast, the `/pages/{id}/children` above would list all direct child content regardless of type, and `/folders/{id}/direct-children` or others cover other types.

All “direct-children” endpoints support common query params like `limit` and `cursor` for pagination. The returned objects include each child’s basic info (ID, title, type, status, etc.). **Permissions**: The user must have view permission on the parent and the children; only children the user can see are returned ⁴⁹.

Classification Level (Data Classification)

Confluence Cloud supports content classification (like marking content as “Official”, “Confidential”, etc. under Data Security policies). These endpoints manage classification levels:

- **GET** `/classification/levels` - **List classification levels**: Returns all classification levels available in the site (e.g. “Public”, “Sensitive”, “Confidential”, etc., depending on your organization’s policy) ⁶⁰. This is essentially the list of defined labels and their meanings.
- **GET** `/spaces/{id}/classification` - **Get space default classification**: Returns the default classification level for a given space ⁶¹. This is the classification applied to new content in that space if not overridden.
- **PUT** `/spaces/{id}/classification` - **Update space default classification**: Sets a new default classification level for the space ⁶². (Requires space admin permission.) For example, you could set a particular space’s default to “Confidential”.
- **DELETE** `/spaces/{id}/classification` - **Delete space default classification**: Removes the custom default classification for the space ⁶³, causing the space to revert to the global default classification.
- **GET** `/pages/{id}/classification` - **Get page classification**: Retrieves the classification level assigned to a specific page ⁶⁴. If a page doesn’t have an explicit classification, this may return the space default or indicate none. (*Draft pages may not support classification; the API notes “not for drafts” in docs* ⁶⁵.)
- **PUT** `/pages/{id}/classification` - **Update page classification**: Sets or changes the classification level of a page ⁶⁶ (e.g. mark a page as “Official”). Requires permission to edit the page.
- **POST** `/pages/{id}/classification/reset` - **Reset page classification**: Resets the page’s classification to the space default (removing any custom classification on that page) ⁶⁷.

- **GET/PUT/POST for Blog Posts, Whiteboards, Databases** – Similarly, the API provides **Get/Update/Reset** endpoints for classification on **blog posts**, **whiteboards**, and **databases** ⁶⁸ ⁶⁹. Their usage is analogous to the page endpoints: for example, **GET** `/blogposts/{id}/classification` returns a blog's classification level ⁷⁰, **PUT** updates it, and **POST** resets it to the space default ⁷¹. Likewise for whiteboards and databases ⁷² ⁷³.

Each “Get ... classification” endpoint returns a simple object with the content's classification label (or an indication if none). “Update” endpoints require a JSON body specifying the new classification level (often just the level's ID or key). **Permissions:** Viewing a classification requires permission to view that content; updating or resetting requires permission to edit the content ⁷⁴ ⁷⁵. The classification level itself might be associated with certain storage requirements (the API docs note that each level may have handling expectations ⁷⁶ ⁷⁷).

(Classification features are typically part of Atlassian's Data Security or Compliance add-ons. These endpoints will have no effect if classification is not enabled.)

Comment (Page/Blog Comments)

Confluence distinguishes **footer comments** (page comments at the bottom) and **inline comments** (comments on highlighted text). The API provides endpoints for retrieving and managing both:

- **GET** `/attachments/{id}/footer-comments` – **List comments on an attachment:** Returns all *footer* comments on a specific attachment (comments left on the attachment's page) ⁷⁸. Similarly:
- **GET** `/blogposts/{id}/footer-comments` – **Comments on a blog post** ⁷⁹, and **GET** `/pages/{id}/footer-comments` – **Comments on a page:** These return the top-level comments on the specified content (not including inline or nested replies). Results are paginated by `limit` / `cursor` ⁷⁹ ⁸⁰. Each comment object includes its `id`, author, creation date, the content of the comment (in storage and Atlas Doc formats), and metadata including its current `resolutionStatus` (open/resolved) for inline comments ⁸¹. **Permissions:** User must have view permission on the content to see its comments ⁵³.
- **GET** `/attachments/{id}/inline-comments` (and `/blogposts/{id}/inline-comments`, `/pages/{id}/inline-comments`): Returns all *inline* comments on the given content ⁸². Inline comments are those anchored to text within the page/blog. Only top-level inline comment threads are returned (each with its own ID and content). Use the comment's ID to fetch replies if needed (see children below). **Permissions:** Same as above – must view the content ⁸³.
- **GET** `/footer-comments/{id}` – **Get a specific comment:** Retrieves a single comment by its ID (works for footer comments; for inline comments, use the inline endpoint or this if you have the ID). Usually, you'll get the comment's content and details.
- **PUT** `/footer-comments/{id}` – **Update a comment:** Updates the content of a comment (e.g., edit the text). The request body should include the new comment body content (usually in Confluence storage format). **Permissions:** User must be the comment author or have permission to edit others' comments (space admin). Returns the updated comment.
- **DELETE** `/footer-comments/{id}` – **Delete a comment:** Deletes the specified comment (footer or inline). This will remove the comment (and its replies) from the page. **Permissions:** Requires permission to delete comments (authors can delete their own; space admins can delete any) ⁸⁴. Returns 204 on success.

- **GET** `/footer-comments/{id}/children` – **Get comment replies**: Returns the immediate replies (children) of a specific comment thread ⁸⁵. Use this to fetch responses to a top-level comment. Similarly, **GET** `/inline-comments/{id}/children` would retrieve replies to an inline comment thread. Pagination supported if many replies.
- **POST** `/footer-comments` – **Create a comment**: Creates a new comment. This **generic endpoint** is used for adding either a top-level comment or a reply. The request body JSON must specify one of `pageId`, `blogPostId`, `attachmentId`, or `customContentId` to indicate where the comment should be added, **and** optionally `parentCommentId` if it's a reply rather than a new thread ⁸⁶ ⁸⁷. It also includes the comment `body` (with a representation type and value). For example, to add a page comment, provide `"pageId": 123` and the body; to reply to another comment, also include `"parentCommentId": 456` ⁸⁶. **Permissions**: Requires permission to comment in that space (and to reply if replying) ⁸⁸. Returns the created comment object with its ID, etc. on success ⁸⁹. (*Apps cannot create inline comments via this – inline comments require an anchor; but if `inlineMarkerRef` is included in properties, an inline comment could potentially be created. However, the Forge/UI normally handles inline anchors.*)
- **POST** `/inline-comments`: (Not explicitly documented, likely creation of inline comments is done via the same `/footer-comments` endpoint by specifying an `inlineMarkerRef` property in the body properties.) The **general creation endpoint** uses the provided context to determine if it's inline or footer. The returned comment will indicate `resolutionStatus` and include an `inlineMarkerRef` if it's an inline thread ⁹⁰.

Summary: To get all comments on a page or blog, you typically call both `/.../footer-comments` (for regular page comments) and `/.../inline-comments` (for inline comments anchored in text). Comments are paginated; each has an `id` for further operations. You can fetch replies for any comment via the children endpoint. **Permissions:** Viewing requires page view permission; editing/deleting comments requires appropriate rights (author or admin) ⁸⁴. Comments are not subject to app data security exemptions (they obey the normal permission rules) ⁹¹ ⁹².

Content (Generic) and ID Conversion

- **POST** `/content/convert-ids-to-types` – **Convert IDs to types**: Takes a list of Confluence content IDs (which on their own are ambiguous in v2) and returns their content types ⁹³. In Confluence Cloud v2, an ID isn't sufficient to know what object it refers to (e.g., a numeric ID could belong to a page, blog, comment, etc.). This endpoint helps migration from v1 by telling you the type for each ID. **Request:** a JSON payload with an array of IDs (and maybe their container context) – the v2 OpenAPI spec indicates an input like `{"ids": [...ids...]}`. The response maps each ID to a `type` such as `"page"`, `"blogpost"`, `"attachment"`, `"inline-comment"`, `"footer-comment"`, etc. ⁹³ ⁹⁴. Notably, v2 splits comments into "inline-comment" vs "footer-comment" types, whereas v1 just called both "comment" ⁹⁵. If an ID is invalid or the user lacks access, it will map to `null` in the response ⁹⁶. **Permissions:** Requires view access to each content item; otherwise that item yields null ⁹⁶. OAuth scope `read:content.metadata:confluence` is required ⁹⁷. This endpoint is especially useful when migrating stored content IDs from API v1 to v2 context.

(The "Content" group in v2 primarily refers to this ID conversion utility. Other content operations are covered under specific types like Page, Blog Post, etc.)

Content Properties

Content Properties are key-value pairs attached to Confluence content (pages, blog posts, etc.) that apps and integrations can use to store custom data. The v2 API provides separate endpoints for each content type's properties:

- **List content properties** (for a given content): e.g. **GET** `/pages/{id}/properties` returns all properties on that page ⁹⁸. Similarly `/blogposts/{id}/properties`, `/attachments/{id}/properties`, `/whiteboards/{id}/properties`, `/databases/{id}/properties`, `/folders/{id}/properties`, `/custom-content/{id}/properties`, and even `/comments/{id}/properties` exist (as indicated by the exhaustive nav list) ⁹⁹ ¹⁰⁰. Each returns an array of property objects with `key`, `value`, and metadata (like creation date and version of the property) ⁹⁹. Supports pagination if a content has many properties.
- **Get content property by key**: e.g. **GET** `/pages/{id}/properties/{propertyKey}` retrieves a specific property on that page by its key ⁹⁸. If found, returns the property object (`id`, `key`, `value`, and a `version` sub-object tracking updates) ¹⁰⁰. If not found, returns 404.
- **POST Create content property**: e.g. **POST** `/pages/{id}/properties` with JSON body (containing a `key` and `value`) creates a new property on that page ⁹⁸. The key must be unique on that content. Returns 201 with the created property.
- **PUT Update content property**: e.g. **PUT** `/pages/{id}/properties/{propertyKey}` updates the value (and increments the version) of an existing property ¹⁰¹. You must supply the new value (and maybe the current version number depending on concurrency control). Returns the updated property.
- **DELETE content property**: e.g. **DELETE** `/pages/{id}/properties/{propertyKey}` removes that property from the content ¹⁰². Returns 204 on success.

These endpoints exist for each content type (page, blog, custom content, etc.) – the only difference is the path. For example, to manage a blog post's property, use `/blogposts/{id}/properties...` with the same pattern ¹⁰³. To manage a comment's property, use `/comments/{id}/properties...` (this allows apps to annotate even comments) ¹⁰⁴ ¹⁰⁵.

Use case: Content properties are often used by apps to store custom flags or data (like workflow status, synced IDs, etc.) on a Confluence content item. They are not visible in the Confluence UI, but accessible via API.

Permissions: The user must have permission to view the content to read its properties, and permission to edit the content to create/update/delete properties ⁹⁹. Some content properties may be set by apps with elevated rights (Connect apps can use app scope to manage them). Data stored here is subject to Confluence's app access rules (unless marked exempt, which by default they are not).

*(In summary, for any content piece X, you can do **GET** `/X/{id}/properties` (list), **GET** `/X/{id}/properties/{key}`, **POST** `/X/{id}/properties`, **PUT** `/X/{id}/properties/{key}`, **DELETE** `/X/{id}/properties/{key}`. Replace X with pages, blogposts, etc. as needed.)*

Custom Content

“Custom Content” refers to new content object types introduced by apps or by Atlassian (beyond pages/blogs). These endpoints allow managing such content:

- **GET** `/custom-content?type={contentType}` – **Get custom content by type**: Lists all custom content of a given type that the user has access to. You must specify a `contentType` (for example, an app-defined content type key). This returns a paginated list of those content items, similar to how pages or blogs are listed. You can filter by space (`spaceId` or `spaceKey` query) to narrow scope.
- **GET** `/custom-content/{id}` – **Get custom content by ID**: Retrieves a specific custom content item's details (title, body, space, status, etc.) by its ID ¹⁰⁶ ¹⁰⁷. The response includes its `id`, `customContentTypeId` (the type key), and content similar to a page (with body in storage format, version info, etc.). **Permissions**: requires view permission on that content's space.
- **POST** `/custom-content` – **Create custom content**: Creates a new piece of custom content. The request JSON should include at least a `title`, a `spaceId` or `spaceKey` (where to create it), the `customContentTypeId` (identifying which type of custom content), and a `body`. **Permissions**: The user must have permission to create content in that space (often this maps to page creation permission if the custom content is page-like). On success, returns the created object (201).
- **PUT** `/custom-content/{id}` – **Update custom content**: Updates an existing custom content item's title or body. Requires providing a new version number in the payload (similar to updating pages) to avoid edit conflicts. **Permissions**: requires edit permission on that content (which typically follows space/page edit rules or app-specific rules). Returns the updated content.
- **DELETE** `/custom-content/{id}` – **Delete custom content**: Deletes the item. It likely moves it to trash if applicable (or permanently removes it if the type doesn't support trash). **Permissions**: requires delete rights (often only space admins or content owners). Returns 204 on success.
- **GET** `/blogposts/{id}/custom-content?type={typeKey}` – **Custom content in a blog**: Lists custom content items of a specific type that are children of a given blog post ¹⁰⁶. For example, if an app attaches custom content to blog posts, this retrieves them. Similarly, **GET** `/pages/{id}/custom-content?type=...` for custom content under a page, and **GET** `/spaces/{id}/custom-content?type=...` for all custom content of that type in a space ¹⁰⁸. These help locate custom content items in context.

Example: Atlassian's “Whiteboards” and “Databases” were initially introduced as custom content types (before getting first-class endpoints). An app could define a custom content “Task Report” or similar. These endpoints allow apps to manage those objects as if they were a distinct content type.

Database (Smart Database pages)

Confluence *Database* is a special content type (similar to a page but representing a database table for storing structured data). The API endpoints for databases:

- **POST** `/databases` – **Create database**: Creates a new database object. The request typically requires a `spaceId` (or key) where the database will reside, a `title` for the database, and possibly an initial structure definition (if applicable). **Permissions**: Requires “Create page” permission

in the space (since a database is a content item in the space) ⁵⁰ . Returns the created database with its `id`, `title`, and other metadata (like an empty schema or default fields).

- **GET** `/databases/{id}` – **Get database**: Retrieves the details of a specific database by ID ¹⁰⁹ . The response includes the database's `id`, `title`, `status` (current), `spaceId`, creation info, and possibly the schema/columns of the database (if the API exposes them). **Permissions**: Requires view permission in the space (same as viewing a page) – otherwise returns 403/404.
- **DELETE** `/databases/{id}` – **Delete database**: Deletes the specified database. This likely moves it to trash (if content trash applies) or permanently deletes it. **Permissions**: Requires permission to delete content in that space (usually space admin or author with delete rights). Returns 204 No Content on success ¹⁰⁹ .

(There is no “list all databases” global endpoint; to find databases, you can use the Space hierarchy or search endpoints. For example, databases may appear in the Get descendants or Get space content results, or you can query by content type using CQL.)

Note: A database's contents (rows/records) are not managed via these endpoints – they are managed within the Confluence UI or an upcoming API. The REST endpoints here treat the database as a content container.

Data Policies (Analytics/Data Residency)

These endpoints relate to **Data Security & Governance** settings on Confluence:

- **GET** `/data/policy` – **Get workspace data policy metadata**: Returns metadata about the site's data policy settings ¹¹⁰ . This likely includes whether data residency or security policies are enabled, what classification levels scheme is in use (if any), etc. For instance, it might tell if classification is mandated or if certain content requires labeling. The response could contain flags like “dataPolicyEnabled”: true, default classification, etc. **Permissions**: Only an app (with proper scopes) or an org admin can typically retrieve this. The Atlassian docs note “Only apps can make this request” for similar endpoints ¹¹⁰ , implying a Forge/Connect app with admin privileges may call it.
- **GET** `/spaces/data/policy` – **Get spaces with data policies**: Returns a list of spaces that have a custom data policy applied ¹¹¹ ¹¹² . For example, spaces where data classification or special retention rules are different from the default. It likely provides space IDs/keys and their specific policy overrides. **Permissions**: Possibly restricted to org/site admins or apps (requiring admin scope), since this is sensitive info.

(In practice, these endpoints are part of Compliance features. Regular users would not use them. They help in auditing which spaces have specific data handling rules.)

Descendants (All Descendant Content)

Where the “Children” endpoints give one level down, **Descendants** endpoints fetch *all levels* of content underneath a specified item:

- **GET** `/whiteboards/{id}/descendants` – **All descendants of a whiteboard**: Likely returns any content nested under that whiteboard (though a whiteboard might not have children normally, so this may usually be empty).

- **GET** `/databases/{id}/descendants` – **All descendants of a database**: Returns all content items contained within the database item. If a database can have pages or other content beneath it in hierarchy, those would appear (including their children's children recursively). The structure is typically returned as a tree or flat list with parent references. Paginated by `limit` / `cursor`.
- **GET** `/embeds/{id}/descendants` – **All descendants of a Smart Link**: Smart Links usually don't have descendants, so this will generally return none. Included for completeness of the API.
- **GET** `/folders/{id}/descendants` – **All descendants of a folder**: Returns the entire content tree under the folder (all nested pages, subfolders, etc.) ¹¹³. This essentially gives everything within that folder in one call (with pagination if large). Useful for exporting or analyzing a whole section.
- **GET** `/pages/{id}/descendants` – **All descendants of a page**: Returns **all** content nested under that page, at any depth ¹¹⁴. This includes child pages, grandchildren, as well as any other content types (folders, whiteboards, etc.) under those. It's a comprehensive listing of the page's hierarchy. Each item might include a `type` or similar to distinguish pages vs others.

The descendants endpoints allow retrieving a full hierarchy snapshot. The response likely is a flat list with each item containing parent/child link info, rather than a structured tree (clients may need to reconstruct the tree from `parentId` fields). **Permissions**: The user must have view permission on the root content and every descendant; items the user cannot see are omitted. Typically, if you can see the parent, you can see descendants unless specific restrictions apply (which Confluence can have via page restrictions – those restricted pages would be absent).

These endpoints are useful for export or for quickly finding all items under a given node without multiple API calls.

Folder

Folders are a new content organizing construct in Confluence Cloud (they act as containers for pages or other content, primarily in the sidebar hierarchy). Endpoints for folders:

- **POST** `/folders` – **Create folder**: Creates a new folder in a specified location. The request JSON likely needs a `spaceId` (or parent `folderId` or parent `pageId`) to indicate where to create it, plus a `title` for the folder. For example, to create a top-level folder in a space, provide the space; to create a sub-folder, provide a parent folder ID. **Permissions**: Requires the same permission as creating a page in that location (i.e., add content permission in space) because a folder is a content entity. Returns the created folder object (with an `id`, `title`, and parent reference).
- **GET** `/folders/{id}` – **Get folder by ID**: Retrieves metadata about a folder (its title, location, and perhaps children count). **Permissions**: Requires view access to the space/parent. Returns the folder's details including its `id`, `title`, `status` (should be "current" unless deleted/trashed), and parent (could be a space or another folder).
- **DELETE** `/folders/{id}` – **Delete folder**: Deletes the folder. This likely moves the folder (and all content within it) to trash. Deleting a folder essentially means all pages and items under it are also trashed as a batch (Confluence likely treats it similar to deleting a page with children). **Permissions**: Requires permission to delete all content in that folder (space admin or the person who could individually delete those pages). In practice, probably only space admins can delete a folder because of the breadth of impact. Returns 204 on success.

(There is no direct update folder endpoint; renaming a folder might be done via the regular update content endpoint if folder were considered a content item, but currently renaming a folder can be done via the UI or possibly via updating its title through a similar endpoint as pages – since the nav suggests no explicit “PUT folder” in v2, folder renames might not be open via REST yet.)

Folders enable organizing pages but do not themselves have a body content. Think of them as empty nodes that just carry a title and hold other items. The API treats them as content for hierarchy purposes.

Label

Labels can be applied to pages, blog posts, attachments, etc., and the API allows retrieving labels and labeled content. Key endpoints:

- **GET** `/attachments/{id}/labels` – **Get labels on an attachment**: Returns all labels applied to the specified attachment ¹¹⁵. Labels have a `name` and a `prefix` (e.g. “global” or “team” label) ¹¹⁶. Similarly:
- **GET** `/blogposts/{id}/labels` – **Labels on a blog post** ¹¹⁷, **GET** `/pages/{id}/labels` – **Labels on a page** ¹¹⁸, **GET** `/custom-content/{id}/labels` – **Labels on custom content** ¹¹⁹. Each returns the set of labels attached to that content. Supports `prefix` query to filter by prefix (e.g., only “team” labels) and `sort` (like alphabetical) ¹²⁰. **Permissions**: Need view permission on the content to see its labels ¹²¹.
- **GET** `/labels` – **Get all labels**: Returns all labels in the site (or possibly all labels the user has access to) ¹²². This can be a large set. Supports pagination (`limit`, `cursor`) and a `prefix` filter to fetch labels starting with a certain string. Useful for building tag clouds or auto-complete for labels. Each label object has `id`, `name`, and `prefix`.
- **GET** `/spaces/{id}/labels` – **Get labels for space**: Returns labels attached to the space itself (space labels, a.k.a. space categories) ¹²³. Spaces can have labels (used to categorize spaces). This endpoint lists those.
- **GET** `/spaces/{id}/content/labels` – **Get labels of space content**: Returns all labels used on any content within the specified space ¹²⁴. In other words, it aggregates labels from pages, blogs, etc. in that space. This is useful to see what tags are common in a space. Pagination is supported since this could be numerous. Only labels on content the user can see are included.

(The API v2 currently does not provide a way to add or remove labels via REST – those operations might still use the older v1 API (`POST /content/{id}/label`). So in v2, labels are read-only. Manage labels through Confluence UI or v1 if needed.)

Label object: Each label has a `name` and a `prefix`. Prefix is usually “global” for user-added labels, or other values like “team” or “category” depending on usage. The combination of prefix and name uniquely identifies a label ¹¹⁶.

Permissions: Anyone who can view the content can view its labels. The list of all labels or space labels requires “can use” site permission (basically any logged-in user) ¹²¹.

Like (Content Likes)

These endpoints let you retrieve who liked content and how many likes there are. In Confluence Cloud, users can like pages, blog posts, and comments.

- **GET** `/blogposts/{id}/likes/count` – **Get like count (blog)**: Returns the number of likes on the specified blog post ¹²⁵. The response is a simple JSON like `{ "count": 5 }` ¹²⁶.
- **GET** `/blogposts/{id}/likes/users` – **Get likers (blog)**: Returns the account IDs of users who liked that blog post ¹²⁷ ¹²⁸. This is paginated if there are many likers. Example response: `{ "results": [ { "accountId": "abc123" }, ... ], "_links": {...} }` ¹²⁹.
- **GET** `/pages/{id}/likes/count` – **Like count (page)**: Number of likes on a page ¹³⁰ ¹³¹.
- **GET** `/pages/{id}/likes/users` – **Likers (page)**: Account IDs of users who liked the page ¹³² ¹³³.
- **GET** `/footer-comments/{id}/likes/count` – **Like count (footer comment)**: Number of likes on a page/blog comment ¹³⁴ ¹³⁵. Comments (both footer and inline) can be liked by users.
- **GET** `/footer-comments/{id}/likes/users` – **Likers (footer comment)**: Who liked a particular footer comment ¹³⁶ ¹³⁷.
- **GET** `/inline-comments/{id}/likes/count` – **Like count (inline comment)** ¹³⁸ ¹³⁹.
- **GET** `/inline-comments/{id}/likes/users` – **Likers (inline comment)** ¹⁴⁰ ¹⁴¹.

All these endpoints simply **retrieve** like information. They do **not** allow setting/clearing a like via v2 (there is no POST/DELETE for likes in v2) – liking/unliking content is typically done via the UI, and programmatic like toggling might still require v1 or not be exposed.

Each “users” endpoint is paginated. The `accountId`s returned can be used to fetch user details via the User API or Atlassian identity API if needed.

Permissions: You need view access to the content or comment to query its likes. If you can see the content, you can see how many liked it and who (there’s no separate restriction on likes). For privacy, the list of user IDs is provided but not user names – you’d need to resolve those via the User API or an admin privilege (this is aligned with GDPR compliance).

Operation (Allowed Operations on Content)

These endpoints provide the set of operations (actions) the current user is permitted to perform on a given content entity. They essentially answer the question “Can the user edit this page? Delete it? Comment on it?” in a structured way.

- **GET** `/attachments/{id}/operations` – **Permitted operations for attachment**: Returns a list of operations the user can do on the given attachment ¹⁴² ¹⁴³. For example, the response might include operations like `"read"`, `"delete"`, `"update"`, each with a `targetType` (indicating what the operation applies to, e.g. content). If the user has no special rights beyond view, it might only list `"read"`.
- **GET** `/blogposts/{id}/operations` – **Operations for blog post**: Similar, returns allowed operations on that blog post ¹⁴⁴ ¹⁴⁵. Common operations include `"read"`, `"update"`, `"delete"`, `"export"`, etc., depending on user’s permissions.

- **GET** `/custom-content/{id}/operations` – **Operations for custom content:** Allowed actions for that custom content item ¹⁴⁶ ¹⁴⁷ . If custom content is treated like a page, it could be edit/delete as well.
- **GET** `/pages/{id}/operations` – **Operations for page:** Returns which actions the user can perform on the page ¹⁴⁸ ¹⁴⁹ . For example, if the user is the page owner or has edit rights, the list will include “update”; if they are a space admin, it will include “delete” and perhaps “administer” (like restrictions) operation. If the page is restricted (read-only for user), it might just show “read”.
- **GET** `/whiteboards/{id}/operations` – **Operations for whiteboard** ¹⁵⁰ ¹⁵¹ .
- **GET** `/databases/{id}/operations` – **Operations for database** ¹⁵² .
- **GET** `/embeds/{id}/operations` – **Operations for Smart Link embed** ¹⁵³ .
- **GET** `/folders/{id}/operations` – **Operations for folder** ¹⁵³ .
- **GET** `/spaces/{id}/operations` – **Operations for space:** Allowed operations in the context of that space for the user ¹⁵⁴ . This might include “administer” if the user is a space admin, or “create” if the user can create content, etc.
- **GET** `/footer-comments/{id}/operations` – **Operations for a footer comment:** E.g., can the user edit or delete that comment (likely returns “update” and “delete” if the user is the author or has admin rights, or empty if they cannot modify it).
- **GET** `/inline-comments/{id}/operations` – **Operations for an inline comment:** Similar idea for inline comments.

The response format is typically an object: `{ "operations": [ { "operation": "<action>", "targetType": "<string>" } , ... ] }` ¹⁵⁵ . Each entry's `operation` might be a key like “read”, “create”, “delete”, “edit” etc., and `targetType` could be “page”, “blogpost”, “comment”, etc., indicating the scope of that operation. Essentially it enumerates what the user is allowed to do.

Use case: Instead of hard-coding permission checks, a client can use these endpoints to decide which UI actions to enable. E.g., if “delete” is present for a page, show a delete button.

Permissions: You need at least read access to call these; if you have no access, you wouldn't be able to call it in the first place (and if you did, it'd 403/404). The returned operations are inherently what you're permitted. This is read-only info.

Page

Pages are the primary content in Confluence. The Page endpoints allow full CRUD and more:

- **GET** `/pages` – **List pages:** Returns all pages the user can see (usually across the site or filtered by query). By default, it returns pages sorted by ID ascending ⁴⁷ . You can apply filters: `spaceId` or `spaceKey` to limit to a space, `title` to search for a title, `type` (though pages are type “page”), `status` (current or trashed), `labels` (filter pages having certain labels), etc. Also supports sorting via `sort` param (e.g. `title-ascending`, `created-date-newest-to-oldest` ¹⁵⁶). Paginates with `limit` and `cursor` . **Permissions:** Only pages the user has view permission for are included ¹⁵⁷ . Each page result includes key fields (id, title, space, status, author, created/updated dates, etc.).
- **POST** `/pages` – **Create page:** Creates a new page. The request JSON must include `spaceId` or `spaceKey` where to create it, a `title` , and a `body` object (usually with representation

"atlas_doc_format" or "storage" and the content value). If creating as a child of another page, include `parentId`. **Permissions:** Requires permission to create pages in the target space (and if `parentId` is given, user must have view access to that parent). On success, returns 201 with the new page object (including its assigned ID, etc.) ^{158 159}.

- **GET** `/pages/{id}` – **Get page by ID:** Retrieves a page's details. By default, returns the latest version. You can use query params to expand content: e.g. `body-format=storage` (or `view`) to get content, or booleans like `includeAttachments`, `includeChildren`, `includeDescendants`, `includeOperations`, etc. (v2 uses separate endpoints for many of those, so `include*` flags might be limited). **Permissions:** Requires view permission on that page ^{160 161}. Returns all page fields, including body (if requested), version info, and maybe embedded lists of operations/permissions if `include-operations` or `include-permissions` is used ^{162 163}.
- **PUT** `/pages/{id}` – **Update page:** Updates an existing page's content. The body of the request should contain the new `title` (or you can use the separate title endpoint) and new `body`. Crucially, it must include a `version` object with an incremented version number (current version + 1) ⁵⁴. If the version is outdated (someone updated in between), the API will reject with conflict – you need the latest version number. You can mark a minor edit via `version.minorEdit = true` if needed. **Permissions:** Requires edit permission on the page ¹⁶⁴. On success, returns the updated page representation. Updating the page creates a new version in Confluence's history.
- **DELETE** `/pages/{id}` – **Delete page:** Deletes the page. This will either trash the page (if trash is enabled) or remove it permanently if it was already trashed. If the page has descendants (child pages), Confluence will typically trash them as well (cascading delete). **Permissions:** Requires permission to delete the page – usually space admin or the page's own administer permission if granted. On success, returns 204. *(Deleting a page via API v2 is an irreversible operation for regular users since v2 doesn't have an "undo" except restoring from trash via the UI or v1 API.)*
- **PUT** `/pages/{id}/title` – **Update page title:** A convenience endpoint to change a page's title without touching its body ^{165 166}. The request body likely just contains the new `title` and a `version` increment. This is equivalent to doing a full update with just the title changed, but is simpler. **Permissions:** Same as editing the page ¹⁶⁷. Use this if you only want to rename a page.
- **GET** `/spaces/{id}/pages` – **Get pages in space:** Returns all pages in the specified space (that the user can view) ¹⁶⁸. Useful for listing or inventory. Supports pagination. It defaults to all pages (perhaps sorted by title or ID). You can combine this with label filters or others by using query params as in `/pages`. **Permissions:** Requires view on space; returns only pages visible to the user.
- **GET** `/labels/{id}/pages` – **Get pages for label:** Retrieves all pages tagged with a given label ¹⁶⁸. Provide the label ID or name. This is similar to a label search specifically for pages. Pagination supported.

(Note: Creating or updating a page via these endpoints uses Atlassian's new document format (ADF) for the body by default. You may specify storage format, but Confluence might expect the Atlas Document format for rich content in v2. The `body-format` query on GET and representation in PUT/POST should be used carefully – many clients use ADF (`atlas_doc_format`) as `representation`.)

Permissions Recap:

Viewing pages: need space/page view permission (if page is restricted, user must be in allowed users).
Editing pages: need space add page permission and no page-level restrictions preventing it. Deleting pages:

need space admin or “Delete” permission on that page (often only admins have that). The API will return 401/403 if permissions are lacking.

Redactions (Content Redaction)

These endpoints support **redacting** text in pages or blog posts – a compliance feature allowing portions of content to be hidden with a marker (for example, to hide sensitive info like PII). **Note:** Requires Atlassian’s Guard (Premium security feature) to be enabled ¹⁶⁹.

- **POST** `/pages/{id}/redact` – **Redact content in page:** Takes a page and a set of instructions for redaction, and produces a new version of the page with specified text replaced by redaction blocks ¹⁷⁰. The request body must specify the ranges of text to redact (likely by pointer IDs or offsets in the content). Each specified redaction will result in a redacted segment (often shown as a black bar in UI). The API responds with a 202 Accepted if the redaction was applied, and returns details of each redaction including a unique UUID that can be used to *restore* that redacted portion later ¹⁶⁹ ¹⁷¹. (Code block content cannot be restored once redacted, per note.) The response’s `RedactionResponse` enumerates the redactions and their IDs ¹⁷². **Permissions:** Requires permission to edit the page (since it creates a new version) ¹⁷³, and specifically the user must be a site admin or have a role allowed to redact (since it’s a high-privilege action). This endpoint is only available to org admins via an app or directly, not to normal users. OAuth scopes needed: `write:page:confluence` (and possibly a special scope for redaction if any) ¹⁷⁴.
- **POST** `/blogposts/{id}/redact` – **Redact content in blog post:** Same as above, but for a blog post ¹⁷⁵. It redacts specified text in the blog content. Requirements and behavior are analogous to the page redaction.

(Currently, the API v2 does not list a direct “restore redaction” endpoint – instead, each redaction has a UUID that could be recorded. Restoration might be done via another admin process or version restore. The main aim here is to programmatically redact sensitive info.)

These endpoints return as **Accepted (202)** because the redaction might be processed asynchronously (especially if a page is large or many redactions). However, in practice Confluence likely applies them quickly and the response includes the results immediately. The `createdAt` and `cleanHistory` fields in the request allow specifying when the redaction is done and whether to remove redacted text from page history as well (the `cleanHistory` boolean indicates if historical versions should also be purged of this text) ¹⁷⁶.

Important: Only available on Premium with an org admin context – normal apps/users won’t have access (the Atlassian documentation explicitly notes apps cannot bypass app access rules for this, meaning it’s not exempt from the usual security – it must be an admin action) ¹⁷⁷.

Smart Link

Smart Links are a type of embedded content (e.g., a preview of an external link like a Google Doc). In Confluence’s new content model, they can be treated as content elements in the page tree (especially when

inserted as independent objects). The API endpoints for Smart Links allow management of standalone smart link content items:

- **POST** `/embeds` **(Smart Link in content tree) – Create Smart Link**: Creates a new Smart Link object in the content tree. The nav shows “**POST Create Smart Link in the content tree**” ¹⁷⁸. This likely requires specifying the parent (spaceId or container page/folder) and the URL or data for the Smart Link. For example, you might provide the `destination` URL and Confluence will create a smart link object that renders that. **Permissions**: similar to creating a page in that space (the user needs permission to add content in that container). Returns the created Smart Link object with its ID.
- **GET** `/embeds/{id}` **– Get Smart Link by ID**: Retrieves details of the Smart Link (its URL, title, status, and any preview metadata Confluence has). **Permissions**: need view access to its container.
- **DELETE** `/embeds/{id}` **– Delete Smart Link**: Removes the Smart Link object (essentially unlinking that embedded content). **Permissions**: need permission to delete content in that container. Returns 204 on success.

(The term “embeds” is used for Smart Links in the API. These endpoints manage Smart Links that exist as independent content items in a space, not ones embedded in page body text. Many Smart Links are just inline in page bodies – those are part of the page content and not manipulated via these endpoints. These endpoints address Smart Links that appear as separate, nestable content items – e.g., a Smart Link card inserted as its own block in the page tree.)

Space

Spaces are containers for content in Confluence. Space endpoints include listing spaces and space creation:

- **GET** `/spaces` **– Get spaces**: Returns all spaces visible to the user ⁴⁷. By default, results are sorted by space ID ascending, which effectively is order of creation, but the docs note “sorted by id ascending” (which might align with alphabetical key if created in certain order, but not guaranteed). You can filter with query params: `ids` (list of space IDs to retrieve specific ones), `keys` (list of space keys), `type` (e.g. “global” or “personal”), `status` (“current” or “archived”), `labels` (spaces that have certain labels), `favorited-by` (spaces a given user has favorited), etc. ^{179 180}. Without filters, it lists all spaces. Supports pagination via `limit` & `cursor`. Each space in results includes: `id`, `key`, `name`, `type` (“global” for regular, “personal” for personal spaces), `status` (current or archived), the `authorId` (creator), `createdAt`, `homePageId` (the ID of the home page), and possibly an icon link and description in different representations ^{181 182}. **Permissions**: Only spaces the user can access are returned (the user must have “Can use” global permission to see any spaces at all, and at least view rights for each space) ¹⁸³.
- **POST** `/spaces` **– Create space**: Creates a new space. **(This endpoint is marked experimental)** ¹⁸⁴. The request JSON should include: `key` (a unique space key, usually 2–10 characters), `name` (space name), and optionally a `description` object and some settings. You can also specify `alias` (for personal spaces typically), `templateKey` if creating via a template, or `createPrivateSpace=true` to immediately restrict it. You can also assign some initial `spacePermissions` via `roleAssignments` array (if supported) ¹⁸⁵. **Permissions**: Requires “Create spaces” global permission ⁵⁰ (usually only granted to admins or allowed users). Also, space keys must be unique and not conflict with existing or reserved keys. On success, returns 201 with the created space details (including its ID and the provided attributes) ¹⁵⁸. If the key is taken or invalid, returns 400.

- **GET** `/spaces/{id}` – **Get space by ID**: Retrieves a space's details by space ID ¹⁶⁰. Alternatively, you might provide the space key in place of ID (the docs often accept either via path or query). The response includes space info similar to the list: id, key, name, type, status, description (both plain text and view HTML if available) ¹⁸⁶, and can optionally include expansions: e.g., `include-icon=true` to get space icon URL ¹⁸⁷, `include-operations`, `include-permissions`, `include-role-assignments`, `include-labels` (these are boolean query params to include those sub-resources) ¹⁸⁸ ¹⁶². **Permissions**: Requires view access to the space (or if it's a personal space, being that user or having admin rights). If the user lacks access, a 403/404 is returned. On success, returns 200 and space details (with nested lists if expansions were requested, such as labels or permissions) ¹⁸⁹ ¹⁶³.

(There is no direct update or delete for spaces in v2. Space settings (name, description) can be changed via Confluence UI or potentially via older API or GraphQL. And deleting spaces is an admin-level operation not exposed in v2 as of now.)

Space Permissions

These endpoints deal with **space-level permissions** (who can do what in a space). They are primarily read-only in v2:

- **GET** `/spaces/{id}/permissions` – **Get space permission assignments**: Returns the permission settings for the given space ¹⁹⁰. This includes both fixed space permissions (like who can view, who can edit, add pages, administer, etc.) and custom content permissions if any. The response likely contains a list of permissions with their subjects (user or group) and granted operations. For example, it might list that group "confluence-users" has "view" and "add content", user X has "administer" etc. ¹⁹¹. **Permissions**: Requires space admin or Confluence admin rights to view detailed permissions, *but* since this is a GET, it might allow a user to see at least who has what (though historically, only admins can list space permissions) ¹⁹². In v2, if a non-admin calls this, they might get a 403. Assume you need to be a space admin or higher.
- **GET** `/space-permissions` – **Get available space permission keys**: Returns the list of all possible space permission types (the operations that can be assigned). This likely enumerates permission keys and their corresponding "operation.target" to use when managing permissions ¹⁹³. For example: "read" on "space", "create" on "page", "delete" on "blogpost", etc. This is used for reference if building a UI for permissions. **Permissions**: Probably open to admins only.

(The v2 API does not yet provide POST/PUT to modify space permissions – those remain in v1. So you cannot add or remove a user's space permission via v2 as of writing. The Community and documentation indicate no v2 for setting space perms yet, except possibly content-level restrictions via the "Operation" or "Space role" endpoints.)

Space Properties

Similar to content properties, spaces can have arbitrary key-value properties (often used by apps to store metadata about a space):

- **GET** `/spaces/{id}/property` – **List space properties**: Returns all properties set on the space. Each property has an `id`, a `key`, a `value`, and version info (if updated). For instance, an app might store a property like `{"key": "homepageLayout", "value": "custom"}` on a space.

This endpoint lists all such properties ¹⁰⁰ (keys are usually app-specific). **Permissions:** Requires space admin or at least space view permission (to read properties – likely view is enough for non-sensitive props).

- **GET** `/spaces/{id}/property/{propertyKey}` – **Get space property:** Returns a specific space property by key. If the property exists, returns 200 with the value (and metadata); if not, 404.
- **POST** `/spaces/{id}/property` – **Create space property:** Adds a new property to the space. Provide a JSON with `key` and `value`. **Permissions:** Must have space admin or perhaps edit rights in space (since it's akin to space settings). Returns 201 on success with the created property.
- **PUT** `/spaces/{id}/property/{propertyKey}` – **Update space property:** Updates the value of an existing property. Provide the new value (and likely the current property version). Returns 200 with updated data.
- **DELETE** `/spaces/{id}/property/{propertyKey}` – **Delete space property:** Removes the property from the space. Returns 204 if successful.

These mirror the content properties usage, just at space level. They are often used to store space configuration by apps (for example, a Publishing app might store “this space is published to URL X” as a property).

Permissions: Typically, reading space properties might be allowed for any user who can view the space (since they're not inherently secret unless the app data is sensitive). Modifying space properties should require space admin rights or an app context with proper scope, because it's like altering space configuration. The API docs don't explicitly state, but it's safest to assume admin-level permission needed for create/update/delete.

Space Roles

Confluence Cloud allows defining custom space roles (beyond the default “admin”, “user”, etc.) and assigning users to them. Space role endpoints let you query and manage these roles:

- **GET** `/space-role` (or `/space-roles`) – **Get available space roles:** Returns all space role definitions in the site ¹⁹⁴. This includes default roles (like “confluence-administrator”, “confluence-user”, which are global though, possibly not listed here) and any custom roles defined (e.g. “Team Lead”, “Project Manager” roles created by an admin). Each role has an `id`, a `name`, and a set of default permissions perhaps. **Permissions:** Must be a site admin to view roles, since normal users can't enumerate all roles.
- **POST** `/space-role` – **Create a space role:** Allows creation of a new custom space role (experimental). The request would include a `name` (role name) and possibly a set of permissions this role grants by default (though permission assignment is usually separate). **Permissions:** Site admin permission required. Returns the created role (`id`, `name`).
- **GET** `/space-role/{roleId}` – **Get space role by ID:** Retrieves details of a specific role (like name, and possibly which permissions it corresponds to, and whether it's default or custom) ¹⁹⁵.
- **PUT** `/space-role/{roleId}` – **Update a space role:** Possibly to rename a custom role or change its description. Cannot edit system roles. **Permissions:** Admin only. Returns updated role.
- **DELETE** `/space-role/{roleId}` – **Delete a space role:** Removes a custom role. Only roles not in use can likely be deleted. **Permissions:** Admin only.

- **GET** `/space-role-assignments` – **Get space role assignments**: Likely endpoint to see which users/groups are assigned to what roles, possibly requiring a space context. However, from the nav: **GET Get space role assignments** ¹⁹⁶ suggests maybe **GET** `/spaces/{id}/roles` or similar: it probably returns for a given space, which users have which roles. In Confluence UI, space admin can see that *Space Role assignments* table (e.g., Joe is in role X, team group in role Y). This endpoint returns that mapping for a space. **Permissions**: Space admin required to view.
- **POST** `/space-role-assignments` – **Set space role assignments**: Likely to assign or change roles in a space. Possibly **POST** `/spaces/{id}/role` or similar with a payload of who gets what roles. The nav line **“POST Set space role assignments”** ¹⁹⁷ implies an endpoint to overwrite the assignments for a space’s roles. This could be used to bulk add/remove users to roles in that space. **Permissions**: Space admin or site admin needed. On success, returns updated assignments.
- **GET** `/space-role-mode` – **Get space role mode**: The nav shows **“GET Get space role mode”** ¹⁹⁸. This might indicate whether the instance is using the new granular space roles feature or classic permissions. Possibly returns a flag if custom roles are enabled (Atlas Camp notes mention a “space permission vs space role mode”). If “role mode” is on, custom roles are in effect. **Permissions**: Admin.

In summary, **Space Roles API** allows admin users to define custom roles and manage membership of those roles per space, which is a newer feature for delegated administration. Regular users typically cannot call these except to maybe list roles in a space they admin.

Task (Inline Task / Action Items)

Confluence has **tasks** (action items with checkboxes in page content). The Tasks API lets you find and update these across pages:

- **GET** `/tasks` – **Get tasks**: Returns all tasks visible to the user, across the site ¹⁹⁹. This can be filtered by various query parameters to target specific tasks:
- **status**: filter by status (`"complete"` or `"incomplete"` tasks) ²⁰⁰.
- **space-id**, **page-id**, **blogpost-id**: filter tasks that belong to a particular space or page or blog ²⁰¹. For example, to get tasks in a certain space, provide `space-id=XYZ`.
- **task-id**: you can list specific tasks by their IDs (if you have a list of task IDs).
- **include-blank-tasks**: a boolean, possibly to include tasks with no assignee or no body text (some tasks might be just checkmarks with no text – “blank” tasks). Default may exclude blank tasks unless this is true ²⁰².
- **body-format**: can specify representation for the content in the task body (`plain` or maybe `atlas_doc`), default likely plain text for the task excerpt.

The response is paginated (`limit` & `cursor`). Each task in results includes: an `id`, `status` (“complete” or “incomplete”), the `body` (text of the task), the `assignee` (if the task was assigned to a user – tasks can be @ assigned), and references to where it is: `pageId` or `blogPostId` (the content that contains it), maybe `spaceId` for convenience, and possibly `createdAt` and `dueDate` if present. There may also be a `resolver` or `completedBy` if completed, and `completedAt`.

Permissions: User must have view access to the content containing the task for it to appear ²⁰³. Essentially, it finds tasks in pages/blogs you can see. If you have no special filter, it's "your tasks across Confluence" if you're using it to find tasks assigned to you – but note, the filtering by assignee is not directly listed. Possibly, if you want tasks assigned to a user, you filter by `assigneeAccountId` (the docs don't explicitly list it above, but it might exist as a param). If not, you'd fetch all tasks and filter client-side by assignee.

- **GET** `/tasks/{id}` – **Get task by ID:** Retrieves a specific task (action item) by its ID ²⁰⁴. The task ID is usually a long numeric or string identifier unique to each action item. This returns the same info as list for that one task. If the task doesn't exist or user can't see it, returns 404.
- **PUT** `/tasks/{id}` – **Update task:** Updates a task's status (and possibly other attributes). Typically, this is used to mark a task as complete or incomplete (toggling the checkbox) ²⁰⁵. The request body might accept a `"status": "complete"` or `"status": "incomplete"` to set the new state, and maybe you can also change the assignee via this API by including an `"assignee"` field with an account ID (the UI allows assigning tasks). The API likely allows updating `status` and `assignee` at minimum. **Permissions:** You need edit permission on the page/blog containing the task to change it (because ticking a task checkbox is essentially editing the page content) ⁸⁸. If you only have view, you cannot update the task. On success, returns the updated task object (or 204 No Content).

(Tasks in Confluence are part of page content. The API treats them as separate resources for convenience, but updating a task actually updates the page. The v2 API abstracts this complexity. Marking a task complete via API will create a new minor version of the page with that checkbox checked.)

Note: The tasks API does not create tasks; tasks are created by adding `[]` checkboxes in page content. So there's no POST `/tasks`. Instead, you create a task by editing a page (via Pages API) and inserting a task list item.

The Tasks endpoints are great for building to-do lists across Confluence or for an integration that pulls all tasks assigned to a particular user (though to get "assigned to me" tasks, you'd filter the results where `assignee.accountId == myId` since API allows filtering by assignee in some way – the documentation snippet didn't show `assignee` param, which might be an omission. In practice, Confluence's legacy endpoint had a `assignee` filter. In v2, maybe tasks have an `assignee` field you filter via `task-id` or by retrieving all and filtering.)

User

User endpoints (in Confluence context) are limited, since user management is mostly via Atlassian user APIs. However, Confluence provides an endpoint to fetch user information by accountId:

- **GET** `/users/{accountId}` – **Get user:** Returns profile information for the specified user (by their Atlassian account ID). This typically includes the `accountId`, `displayName`, maybe `publicName`, profile picture links, and possibly status (active/inactive) ²⁰⁶. (Email is not provided for privacy, unless the caller is an admin with scope to get email via a separate API – Confluence REST doesn't return email.) **Permissions:** The calling user must have permission to view user information – usually any logged-in user can fetch another user's basic profile (display name and avatar) ²⁰⁷. However, if privacy mode is strict, some info might be hidden except to admins.

- **GET** `/users/current` (or `/user/current`) – **Get current user**: Returns details of the authenticated user themselves. This is often provided in v1 (`/wiki/rest/api/user/current`). In v2, it might be `/users/me` or similar. If implemented, it returns the same data as above for the current user's account.
- **GET** `/users?accountId={id}` or `?email={email}`: In v1, you could query by username or userKey. In v2, to find a user by email or name is not possible for privacy – you must have the accountId. So likely the only lookup is by accountId (except maybe for your own user via `/me`). *(The nav for v2 doesn't explicitly list user endpoints, but the Postman collection index mentions "User" and "Version" and "Whiteboard" at the end ²⁰⁸. It's safe to assume a GET by accountId is available.)*

There are **no write endpoints for users** in Confluence Cloud API v2 – user administration (creating or removing users, etc.) is handled in Atlassian admin or via SCIM. The Confluence API focuses on retrieving user info for mentions, assignments, etc.

Example use: If you have an accountId from a task assignee field, you can call `/users/{id}` to get that user's name and avatar URL to display in your app.

Permissions: Generally, all users can get basic info of other users (display name) unless privacy settings hide it (in which case maybe only an admin can). The API will return what Confluence displays – often **publicName** (which might be "Anonymous" if the user set their profile to private). So be aware of GDPR mode: if enabled, you might only get an anonymized name for others.

Version (Content Version History)

These endpoints allow retrieving version history of pages and blog posts:

- **GET** `/pages/{id}/versions` – **List page versions**: Returns the list of historical versions for the specified page, in descending order (most recent first) or ascending (depending on default). Each version item includes `number`, `minorEdit` flag, `message` (edit comment), `createdAt` timestamp, and `authorId` of the user who made that version ²⁰⁹. The current version is also listed. Pagination is supported if a page has many versions (use `limit` and `cursor`).
- Permissions:** Need view permission on the page; if the page is restricted and the user can't view historical versions (generally if you can view the page you can view its history, unless an admin hid certain versions), it returns what's allowed.
- **GET** `/pages/{id}/versions/{versionNumber}` – **Get specific page version**: Retrieves the content of a particular version of the page. You can get the old content body (likely in storage format) and metadata. This is essentially "View page history" via API. **Permissions:** Requires view access to the page (and not restricted from seeing that version). Returns 200 with the version's data. The body might be accessible through an expand (e.g., `body-format=storage` to get that version's storage content).
- **POST** `/pages/{id}/versions` – **Restore page version**: *This may or may not exist in v2.* (The Atlassian developer blog hints at restoring via a POST in v1 ²¹⁰, but in v2 it's not confirmed). If implemented, it would take a `versionNumber` in the request body and create a new current version that duplicates that historical version (effectively reverting the page). In absence of a documented v2 restore, you might need to GET the old version content and then PUT it as a new update to simulate restore.

- **DELETE** `/pages/{id}/versions/{versionNumber}` - **Delete page version:** *Not explicitly documented in v2.* Possibly not provided, as cleaning history might remain an admin UI task. (In v1 it existed for Data Center only ²¹¹ ²¹².)
- **GET** `/blogposts/{id}/versions` and **GET** `/blogposts/{id}/versions/{versionNumber}` would be analogous for blog post history if implemented. Blogs do have version history (e.g., if edited multiple times). It's likely available similarly to pages.

The **Version** group in Postman implies endpoints around versions exist ²⁰⁸.

Example: Using these, you can build a “page history” view or get the difference between versions (though differences themselves are not directly given, you'd retrieve two versions and compare content).

Permissions: Only space/page admins or users with view rights can fetch versions. Even if a page is open, if specific historical version content was redacted or hidden, the user might not get it unless admin (but Confluence Cloud doesn't usually hide old versions except via redaction or deletion).

Note: When updating a page via PUT, providing an outdated version number yields a 409 Conflict. So one often GETs the page (to find current version), then PUT with version+1. The version endpoints could also be used to get the latest version number (by looking at the first item in `/versions`). But simply GET page (which includes a `version.number`) is easier for that use.

Conclusion: This comprehensive documentation covers Confluence Cloud REST API v2 endpoints for all major resource types – from content like Pages, Blog Posts, Attachments, Comments and their relationships (ancestors, descendants) to spaces and administrative features (space permissions, roles, data policy), as well as utility endpoints (ID conversion, search by labels, tasks, etc.). An AI agent can use this information to interact with Confluence: for example, creating or updating pages, enumerating content, managing attachments and comments, fetching user info, and enforcing or checking permissions via the Operations API. All responses use standard JSON; create/update calls usually require JSON bodies as described. Remember to include proper **authentication** (such as an API token in Basic Auth or OAuth 2.0 Bearer token) with each request, and the base URL is `https://your-domain.atlassian.net/wiki/api/v2/...`. The agent should also handle pagination (check for `_links.next` in responses to fetch additional results) ¹⁷ ²¹³ and be mindful of rate limits. All actions are subject to Confluence permissions and scopes, so the agent must have appropriate scopes (as noted for various endpoints) when operating within an app context.

Sources: The details above are based on the official Atlassian Confluence Cloud REST API v2 documentation ²¹⁴ ²¹⁵ and relevant Atlassian developer community insights.

¹ ² ³ ⁴ ⁵ ⁶ ⁷ ⁸ ⁹ ¹⁰ ²⁰⁶ ²⁰⁷ The Confluence Cloud REST API
<https://developer.atlassian.com/cloud/confluence/rest/v2/api-group-admin-key/>

¹¹ ¹² ¹⁴ ¹⁵ ¹⁶ ¹⁷ ¹⁸ ¹⁹ ²⁰ ²¹ ²² ²³ ²⁴ ²⁵ ²⁶ ²⁷ ²⁰⁹ ²¹³ The Confluence Cloud REST API
<https://developer.atlassian.com/cloud/confluence/rest/v2/api-group-attachment/>

13 48 156 **Arcade**

<https://docs.arcade.dev/en/resources/integrations/productivity/confluence>

28 29 30 31 32 33 34 35 36 37 38 39 **The Confluence Cloud REST API**

<https://developer.atlassian.com/cloud/confluence/rest/v2/api-group-app-properties/>

40 41 42 43 44 45 **The Confluence Cloud REST API**

<https://developer.atlassian.com/cloud/confluence/rest/v2/api-group-ancestors/>

46 55 56 **The Confluence Cloud REST API**

<https://developer.atlassian.com/cloud/confluence/rest/v2/api-group-blog-post/>

47 49 50 51 157 158 159 160 161 162 163 179 180 181 182 183 184 185 186 187 188 189 215 **The Confluence Cloud REST API**

<https://developer.atlassian.com/cloud/confluence/rest/v2/api-group-space/>

52 53 78 79 81 82 83 84 85 86 87 88 89 90 91 92 **The Confluence Cloud REST API**

<https://developer.atlassian.com/cloud/confluence/rest/v2/api-group-comment/>

54 164 165 166 167 168 **The Confluence Cloud REST API**

<https://developer.atlassian.com/cloud/confluence/rest/v2/api-group-page/>

57 58 59 **The Confluence Cloud REST API**

<https://developer.atlassian.com/cloud/confluence/rest/v2/api-group-children/>

60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 **The Confluence Cloud REST API**

<https://developer.atlassian.com/cloud/confluence/rest/v2/api-group-classification-level/>

80 115 116 117 118 119 120 121 122 123 124 **The Confluence Cloud REST API**

<https://developer.atlassian.com/cloud/confluence/rest/v2/api-group-label/>

93 94 95 96 97 **The Confluence Cloud REST API**

<https://developer.atlassian.com/cloud/confluence/rest/v2/api-group-content/>

98 99 100 101 102 103 104 105 **The Confluence Cloud REST API**

<https://developer.atlassian.com/cloud/confluence/rest/v2/api-group-content-properties/>

106 107 108 **The Confluence Cloud REST API**

<https://developer.atlassian.com/cloud/confluence/rest/v2/api-group-custom-content/>

109 **The Confluence Cloud REST API**

<https://developer.atlassian.com/cloud/confluence/rest/v2/api-group-database/>

110 111 112 **The Confluence Cloud REST API**

<https://developer.atlassian.com/cloud/confluence/rest/v2/api-group-data-policies/>

113 114 **The Confluence Cloud REST API**

<https://developer.atlassian.com/cloud/confluence/rest/v2/api-group-descendants/>

125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 **The Confluence Cloud REST API**

<https://developer.atlassian.com/cloud/confluence/rest/v2/api-group-like/>

142 143 144 145 146 147 148 149 150 151 152 153 154 155 **The Confluence Cloud REST API**

<https://developer.atlassian.com/cloud/confluence/rest/v2/api-group-operation/>

169 170 171 172 173 174 175 176 177 **The Confluence Cloud REST API**

<https://developer.atlassian.com/cloud/confluence/rest/v2/api-group-redactions/>

178 **The Confluence Cloud REST API**

<https://developer.atlassian.com/cloud/confluence/rest/v2/api-group-smart-link/>

190 **The Confluence Cloud REST API**

<https://developer.atlassian.com/cloud/confluence/rest/v2/api-group-space-permissions/>

191 192 193 **The Confluence Cloud REST API**

<https://developer.atlassian.com/cloud/confluence/rest/v1/api-group-space-permissions/>

194 195 196 197 198 **The Confluence Cloud REST API**

<https://developer.atlassian.com/cloud/confluence/rest/v2/api-group-space-roles/>

199 200 201 202 203 204 205 **The Confluence Cloud REST API**

<https://developer.atlassian.com/cloud/confluence/rest/v2/api-group-task/>

208 **The Confluence Cloud REST API v2 | Documentation - Postman**

<https://www.postman.com/api-reference-library/documentation/42654973-3182be8e-99e9-4dec-8b51-e7ec0d6ca3f8>

210 211 212 **The Confluence Cloud REST API**

<https://developer.atlassian.com/cloud/confluence/rest/v1/api-group-content-versions/>

214 **The Confluence Cloud REST API**

<https://developer.atlassian.com/cloud/confluence/rest/v2/intro/>